

分治法

7.1 主定理

分治法是一种常用的算法设计策略。它将一个问题分解为若干个规模较小的子问题，递归地解决这些子问题，然后将它们的解合并为原问题的解。读者在基础编程以及算法课程中已经学习过一些经典的分治算法，例如归并排序，因此这里我们就不需要通过例子来介绍分治法了。我们将首先介绍分治法的时间复杂度分析方法，然后再介绍一些经典的分治算法，并使用介绍的方法来分析它们的时间复杂度。通常而言，分治法的时间递推式具有如下形式：

$$T(n) = aT(n/b) + f(n).$$

其中 a 是子问题个数， b 是子问题规模的收缩因子， $f(n)$ 包括本次调用中除递归调用以外的全部工作：划分输入、准备参数、合并答案等。此外，接下来的对数都默认以 2 为底。

7.1.1 代入法

第一种方法美其名曰代入法，其实就是猜证，显得有些耍无赖，但这种方法在实际应用中十分便捷（如果能猜对的话）。代入法一般分为两步：（1）猜测解的形式；（2）用数学归纳法选择常数，并证明猜测的界成立。至于如何猜测一个好的界，可以根据一些熟悉的结论进行推广，也可以先猜较松的上下界，再往中间逼近。

例题 7.1. 设 $T(1) = 1$, $n \geq 2$ 时 $T(n) = 2T(\lfloor n/2 \rfloor) + n$ 。证明 $T(n) = O(n \log n)$ 。

解答. 首先猜测 $T(n) = O(n \log n)$, 要证明: 存在一个固定的 c , 使所有 $n \geq 2$ 都满足 $T(n) \leq cn \log n$ 。

先看 $n = 2, 3$: $T(2) = 4, T(3) = 5$, 取 $c \geq 2$ 即可。对 $n \geq 4$, 使用归纳假设:

$$T(n) \leq 2c \lfloor n/2 \rfloor \log \lfloor n/2 \rfloor + n \leq cn \log(n/2) + n = cn \log n - (c-1)n \leq cn \log n.$$

□

然而，如果我们猜测 $T(n) \leq cn$, 代入只能得到 $T(n) \leq (c+1)n$ 。虽然右边仍写得成 $O(n)$, 但此时常数改变了，归纳就没有成立。这是因为每递归一层都需要换一个常数，那么递归 $\log n$ 层后，常数就会变成 $O(\log n)$ 级别，因此不再是常数。这提示我们，在使用代入法时，要注意常数在整个归纳过程中保持不变。

例题 7.2. 考虑 $T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \log n$, 设 $T(1) = T(2) = 1$ 。证明 $T(n) = O(\log n \log \log n)$ 。

解答. 先考虑一类特殊的 n 的取值 $n = 2^{2^j}$. 令 $m = \log n$, $S(m) = T(2^m)$, 有

$$S(m) = 2S(m/2) + m.$$

此时每次开平方的结果都是整数。根据例 7.1, 可得 $T(n) = O(\log n \log \log n)$ 。

根据递推式不难得到 $T(n)$ 单调不减。对任意 $n \geq 4$, 取不超过 n 的最大特殊输入 $u = 2^{2^j}$ 。下一个特殊输入恰好是 u^2 , 所以 $u \leq n < u^2$, 故 $T(u) \leq T(n) \leq T(u^2)$ 。由于 $\log(u^2) = 2\log u$, $\log \log(u^2) = \log \log u + 1$, 两端的 $\log n \log \log n$ 只相差常数倍, 因此利用 $T(n) \leq T(u^2)$, 一般输入也满足 $T(n) = O(\log n \log \log n)$ 。□

事实上, 上面的证明也说明了为什么分析递推式时, 常常只考虑能够连续整除的输入: 只需要相邻的可以整除的输入的时间复杂度只相差常数倍, 就可以把一般输入夹在中间, 得到同阶的渐近结果。因此在后文的分析中, 我们统一按能够连续整除的规模计算, 不再逐次讨论取整。

例题 7.3. 设 $T(1) = t > 0$, $n \geq 2$ 时 $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$ 。证明 $T(n) = O(n)$ 。

解答. 我们猜测它的时间复杂度为 $O(n)$ 。如果直接用 $T(n) \leq cn$ 作归纳假设, 代入后只能得到 $cn + 1$, 比要证明的上界多出 1。可以把归纳假设加强为 $T(n) \leq cn - d$, 其中 d 是待定的正常数。这样, 两个子问题各减去一个 d , 代入后得到

$$T(n) \leq c\lfloor n/2 \rfloor - d + c\lceil n/2 \rceil - d + 1 = cn - 2d + 1.$$

要使它不超过 $cn - d$, 只需 $d \geq 1$ 。再取 $c \geq t + d$, 就能保证初值 $T(1) = t \leq c - d$, 归纳证明也就完成了。□

事实上, 用后面第 7.1.2 节介绍的递归树方法, 可以证明上述例题给出的时间复杂度也是下界, 因此都是紧的。

7.1.2 递归树方法

递归树方法就是把递归求解的过程画成一棵树, 再通过这棵树计算总时间。我们直接来看一个例子。设 $T(n) = 3T(n/4) + cn^2$, 并令 $T(1) = t$, 其中 $c, t > 0$ 。这个递推式表示解决一个规模为 n 的问题, 需要递归解决三个规模为 $n/4$ 的子问题, 还要花费 cn^2 的时间完成分解、合并等操作。我们把原问题画成根结点, 三个子问题画成它的三个孩子。每个子问题又产生三个更小的子问题, 就继续向下画, 直到规模变为 1, 递归停止。

图 7.1 中, 根结点标出的 cn^2 是处理原问题时分解、合并等操作所需的时间。三个孩子分别标出 $c(n/4)^2$, 表示处理三个子问题时相应操作所需的时间; 更深的递归调用则继续向下展开。最后, 每个叶子按初值标出 t 。这样, 整个计算过程所花费的时间都分配到了这些结点上, 将它们相加就是 $T(n)$ 。

为了计算这个和, 先把同一层的时间相加。根位于第 0 层, 每向下一层, 子问题个数变为原来的 3 倍, 规模变为原来的 $1/4$ 。所以第 i 层有 3^i 个规模为 $n/4^i$ 的子问题; 只要这一层尚未停止递归, 各结点上标出的时间之和就是 $3^i \cdot c(n/4^i)^2 = cn^2(3/16)^i$ 。由于规模每次变为原来的 $1/4$, 故经过 $\log_4 n$

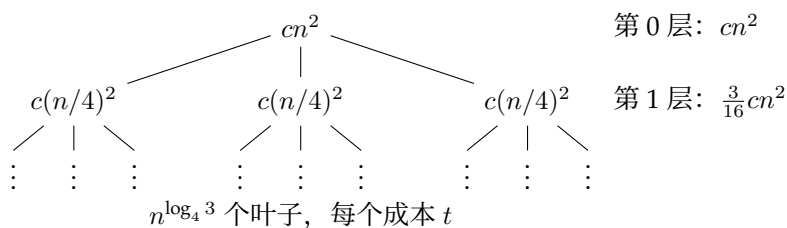


图 7.1 $3T(n/4) + cn^2$ 的递归树。

层后降为 1, 此时共有 $3^{\log_4 n} = n^{\log_4 3}$ 个叶子, 每个需要时间 t 。因此总时间为

$$\begin{aligned} T(n) &= cn^2 \sum_{i=0}^{\log_4 n - 1} (3/16)^i + tn^{\log_4 3} \\ &= cn^2 \frac{1 - (3/16)^{\log_4 n}}{1 - 3/16} + tn^{\log_4 3} = \frac{16c}{13} (n^2 - n^{\log_4 3}) + tn^{\log_4 3} = O(n^2). \end{aligned}$$

又注意到仅根结点就需要 cn^2 的时间, 这又给出了 $\Omega(n^2)$ 的下界, 因此 $T(n) = \Theta(n^2)$ 。

再看一个不能等分的例子: $T(n) = T(n/3) + T(2n/3) + cn$, 且 $T(1) = t > 0$ 。

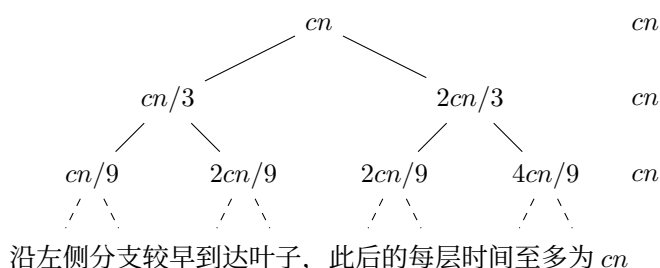


图 7.2 $T(n/3) + T(2n/3) + cn$ 的前三层。

先看还没有出现叶子的前几层。一次调用 $T(x)$ 产生 $T(x/3)$ 和 $T(2x/3)$, 两个参数之和仍为 x , 所以同一层所有调用的参数之和仍为 n , 这一层所需时间就是 cn 。最早出现的叶子位于深度 $\lceil \log_3 n \rceil$, 因此前 $\lceil \log_3 n \rceil$ 层每层都需要 cn 时间, 得到 $\Omega(n \log n)$ 的下界。

再看上界。每次递归后, 参数至多变为原来的 $2/3$, 所以最多经过 $\lceil \log_{3/2} n \rceil$ 层就会停止。一部分调用停止后, 后续每层的时间只会减少 (小于 cn), 因此所有非叶结点合计需要 $O(n \log n)$ 时间。

还要加上叶子的时间。设非叶结点共有 m 个, 这些结点上的总时间为 $O(n \log n)$, 并且每个结点的时间至少为常数 c , 所以它们的个数也满足 $m = O(n \log n)$ 。这棵树的每个非叶结点都有两个孩子, 因此叶子恰有 $m + 1$ 个: 起初只有根这一个叶子, 此后每展开一个结点, 就用两个孩子替换一个叶子, 叶子数增加 1。每个叶子只需固定时间 t , 故叶子的总时间为 $t(m + 1) = O(n \log n)$ 。加上非叶结点的时间, 并结合前面的下界, 得到 $T(n) = \Theta(n \log n)$ 。

也可以用代入法证明上界。取归纳假设 $T(n) \leq dn \log n + en$, 其中 d, e 都是正常数。令 $\eta = \log_2 3 - 2/3 > 0$, 代入后得到

$$\begin{aligned} T(n) &\leq d(n/3) \log(n/3) + d(2n/3) \log(2n/3) + en + cn \\ &= dn \log n + en - (d\eta - c)n. \end{aligned}$$

取 $d \geq c/\eta$, 就得到所需的归纳上界, e 的取值根据初值讨论即可, 不再赘述。

7.1.3 主定理

上述两个例子都有 $\Theta(\log n)$ 层，但每层花费的时间不同。第一个例子中，第一层需要 cn^2 ，后续非叶层的时间都只有上一层的 $3/16$ ，所以这些层相加也只是第一层的常数倍，计入叶子后，总时间仍为 $\Theta(n^2)$ 。第二个例子则有 $\Theta(\log n)$ 层，每层都需要 cn ，这些时间累加后就是 $\Theta(n \log n)$ 。因此两个例子的时间复杂度主要决定的因素不同：第一个例子主要取决于根结点的时间，第二个例子则由层数和每一层的时间共同决定。

主定理 (Master Theorem) 把这种逐层求和的结果归纳成了通用结论。对于 $T(n) = aT(n/b) + f(n)$ ，它根据各层时间的变化，判断总时间主要由叶子决定、由根附近的几层决定，还是需要把同等重要的各层累加起来。

定理 7.4 (主定理). 设 $a \geq 1, b > 1$ 为常数, $f(n) \geq 0$, 当 $n \leq n_0$ 时停止递归, 且在此终止区间内 $T(n) = \Theta(1)$, 其中 n_0 是固定正常数; 当 $n > n_0$ 时, 递推式为 $T(n) = aT(n/b) + f(n)$ 。

1. 若存在常数 $\varepsilon > 0$, 使 $f(n) = O(n^{\log_b a - \varepsilon})$, 则 $T(n) = \Theta(n^{\log_b a})$;
2. 若 $f(n) = \Theta(n^{\log_b a})$, 则 $T(n) = \Theta(n^{\log_b a} \log n)$;
3. 若存在常数 $\varepsilon > 0$, 使 $f(n) = \Omega(n^{\log_b a + \varepsilon})$, 且对某个固定的 $0 < c < 1$ 和所有足够大的 n 都有 $af(n/b) \leq cf(n)$, 则 $T(n) = \Theta(f(n))$ 。

证明. 不失一般性地, 可以取 $n_0 = 1$: 若 $n_0 > 1$, 将 n_0 降到 1 需要原来的叶子继续向下递归。由于每次将输入大小除以固定常数 $b > 1$, 而 n_0 也是固定常数, 故每个原来的叶子最多再展开常数 k 层, 得到的子树最多包含 $1 + a + a^2 + \dots + a^k$ 次调用。这个数是常数, 并且子树中每次调用只需常数时间, 所以整棵子树的总时间仍为 $\Theta(1)$ 。原来每个叶子的时间为 $\Theta(1)$, 展开后整棵子树的时间仍为 $\Theta(1)$, 因此所有叶子对应的总时间只相差常数倍, 故整个算法的渐近阶也就不变。故不失一般性, 以下设 $n_0 = 1$ (之后的定理也不再单独讨论停止条件)。

此时递归树的深度为 $\log_b n$, 叶子数为 $a^{\log_b n} = n^{\log_b a}$, 所以叶子总成本为 $\Theta(n^{\log_b a})$ 。第 i 层有 a^i 个规模为 n/b^i 的子问题, 该层除递归以外的成本为 $a^i f(n/b^i)$ 。把叶子和各个非叶层相加, 得到

$$T(n) = \Theta(n^{\log_b a}) + \sum_{i=0}^{\log_b n - 1} a^i f(n/b^i). \quad (7.1)$$

情况 1 中, 存在常数 C , 使 $f(n) \leq Cn^{\log_b a - \varepsilon}$ 。由于 $a = b^{\log_b a}$, 第 i 层的成本至多为 $a^i C(n/b^i)^{\log_b a - \varepsilon} = Cn^{\log_b a - \varepsilon} b^{\varepsilon i}$ 。又有 $n^{\log_b a - \varepsilon} \sum_{i=0}^{\log_b n - 1} b^{\varepsilon i} = O(n^{\log_b a - \varepsilon} b^{\varepsilon \log_b n}) = O(n^{\log_b a})$, 再加上叶子的 $\Theta(n^{\log_b a})$ 成本, 就得到 $T(n) = \Theta(n^{\log_b a})$ 。

情况 2 中, 第 i 层成本为 $a^i \Theta((n/b^i)^{\log_b a}) = \Theta(n^{\log_b a})$, 与 i 无关。由于共有 $\log_b n$ 个非叶层, 所以总成本为 $\Theta(n^{\log_b a} \log n)$, 叶子成本也包含在这个结果中。

情况 3 中, 正则条件 $af(n/b) \leq cf(n)$ 表示下一层的成本至多是上一层的 c 倍, 由此成本之和至多为 $f(n)(1 + c + c^2 + \dots) = \frac{f(n)}{1-c}$ 。而叶子成本为 $O(n^{\log_b a})$, 条件 $f(n) = \Omega(n^{\log_b a + \varepsilon})$ 保证它们也包含在 $O(f(n))$ 中。因此总成本为 $O(f(n))$ 。注意根结点本身就要花费 $f(n)$, 给出相同量级的下界, 故 $T(n) = \Theta(f(n))$ 。□

实际上主定理并不需要死记硬背, 无论什么形式的主定理, 其实关键都在于比较 $n^{\log_b a}$ 和 $f(n)$ 之间的关联, 如果前者大, 则前者“掌控了”整个时间复杂度, 所以时间复杂度就是 $T(n) = aT(n/b)$

对应的复杂度, 这也很符合直观, 因为此时 a 比较大, 分叉比较多, 树比较大 (结合递归树方法), 所以更大的复杂度会落在叶子上; 反之后者大则每一层的复杂度“掌控了”整个时间复杂度, 故整体时间就是 $f(n)$ 级别的, 这也就是主定理这一名字的含义。

例题 7.5. 设 $T(n) = T(\lfloor n/2 \rfloor) + n(2 - \cos n\pi)$, $T(1)$ 为正常数。判断附加成本是否满足主定理情况 3 的正则条件, 并求 $T(n)$ 的渐近紧界。

解答. 取 $n = 4k + 2$, 则 $f(n) = n$, 而 $f(n/2) = 3n/2$, 因此它不可能对所有充分大的整数都满足 $f(\lfloor n/2 \rfloor) \leq cf(n)$, 其中 $c < 1$ 。

当然这个例子仍然可以直接计算, 因为对所有整数 n 都有 $n \leq f(n) \leq 3n$ 。根据递归树方法可知成本之和至多为常数的叶子成本加上 $3n + 3n/2 + 3n/4 + \cdots \leq 6n$, 得到 $O(n)$ 的上界。又因为根结点的成本至少为 n , 所以 $T(n) = \Theta(n)$ 。□

例题 7.6. 分别求解 $T(n) = 4T(n/2) + n/\log n$ 和 $T(n) = 2T(n/2) + n/\log n$, 其中 $T(1)$ 和 $T(2)$ 为正常数。

解答. 第一问的 $a = 4, b = 2$, 因此 $\log_b a = \log_2 4 = 2$ 。附加成本满足 $n/\log n = O(n^{2-1})$, 由主定理情况 1 得 $T(n) = \Theta(n^2)$ 。

第二问的 $a = b = 2$, 附加成本与 $n^{\log_b a} = n$ 只差一个对数因子, 主定理的三种情况都不能直接使用, 故使用递归树方法。在树的深度 i 处, 有 2^i 个规模为 $n/2^i$ 的子问题, 所以这一层的成本为 $2^i \frac{n/2^i}{\log(n/2^i)} = \frac{n}{\log_2 n - i}$ 。非叶层的 i 从 0 取到 $\log_2 n - 2$ 。把 $j = \log_2 n - i$ 代入, 再计入叶子的线性成本, 得到

$$T(n) = \Theta(n) + n \sum_{j=2}^{\log_2 n} \frac{1}{j}.$$

调和和式满足 $\sum_{j=1}^{\log_2 n} 1/j = \Theta(\log(\log_2 n))$, 例如可以用

$$\int_1^{\log_2 n + 1} \frac{dx}{x} \leq \sum_{j=1}^{\log_2 n} \frac{1}{j} \leq 1 + \int_1^{\log_2 n} \frac{dx}{x}$$

同时估计上下界。因此 $T(n) = \Theta(n \log \log n)$ 。□

相邻层成固定比例的形式

在前面的例子 $T(n) = 3T(n/4) + cn^2$ 中, 下一层的时间总是上一层的 $3/16$ 。如果一般递推式 $T(n) = aT(n/b) + f(n)$ 也具有这种固定比例, 就可以直接使用等比数列求和。

定理 7.7. 设 $a \geq 1, b > 1, c > 0$ 为固定常数, $f(n) > 0$, 且 $T(1) = \Theta(1)$; 当 $n > 1$ 时, 有

$$T(n) = aT(n/b) + f(n), \quad af(n/b) = cf(n).$$

则

$$T(n) = \begin{cases} \Theta(n^{\log_b a}), & c > 1, \\ \Theta(f(n) \log_b n), & c = 1, \\ \Theta(f(n)), & 0 < c < 1. \end{cases}$$

证明. 等式 $af(n/b) = cf(n)$ 表明, 下一层 a 个结点的时间之和是当前结点时间的 c 倍。因此, 从根开始, 各个非叶层的时间依次为 $f(n), cf(n), c^2f(n), \dots$, 它们相加为

$$f(n) \sum_{i=0}^{\log_b n - 1} c^i.$$

同时, 将 $f(n) = (a/c)f(n/b)$ 反复代入 $\log_b n$ 次, 可得 $f(n) = (a/c)^{\log_b n} f(1)$, 故 $f(n)c^{\log_b n} = a^{\log_b n} f(1)$ 。现在分别考虑 c 的大小:

- 如果 $c > 1$, 非叶结点的总时间与 $f(n)c^{\log_b n} = a^{\log_b n} f(1)$ 同阶, 加上叶子的时间, 得到 $\Theta(n^{\log_b a})$ 。
- 如果 $c = 1$, 每个非叶层都需要 $f(n)$ 的时间, 共有 $\log_b n$ 个非叶层, 所以它们的总时间为 $f(n) \log_b n$ 。又由 $f(n) = a^{\log_b n} f(1)$ 可知, 叶子的总时间为 $\Theta(f(n))$ 。将两部分相加, 得到 $T(n) = \Theta(f(n) \log_b n)$ 。
- 如果 $0 < c < 1$, 等比和不超过 $1/(1-c)$, 所以非叶结点的总时间与 $f(n)$ 同阶。又由 $a^{\log_b n} = f(n)c^{\log_b n}/f(1) = O(f(n))$ 可知, 叶子的时间也包含在这一上界中, 因此总时间为 $\Theta(f(n))$ 。

□

允许对数幂的推广

如果递推式为 $T(n) = 2T(n/2) + n \log n$, 附加成本比 $n^{\log_2 2} = n$ 大一个对数因子, 主定理的三种情况无法直接处理, 并且无法找到常数 c 使得 $af(n/b) = cf(n)$ 成立, 因此需要新的判别方法。下面的定理考虑附加成本为 $\Theta(n^k(\log n)^p)$ 形式时的情况, 其中 p 允许为负数。

定理 7.8. 设 $a \geq 1, b > 1, k \geq 0$ 为常数, p 为任意固定实数, 且 $T(1) = \Theta(1)$, 其余输入满足

$$T(n) = aT(n/b) + \Theta(n^k(\log n)^p).$$

此时有

$$T(n) = \begin{cases} \Theta(n^{\log_b a}), & a > b^k, \\ \Theta(n^k(\log n)^{p+1}), & a = b^k, p > -1, \\ \Theta(n^k \log \log n), & a = b^k, p = -1, \\ \Theta(n^k), & a = b^k, p < -1, \\ \Theta(n^k(\log n)^p), & a < b^k. \end{cases}$$

证明. 先看 $a = b^k$ 的情况。深度 i 处的成本为 $a^i \Theta((n/b^i)^k (\log(n/b^i))^p) = \Theta(n^k (\log_b n - i)^p)$, 其中 a^i 与分母中的 b^{ik} 抵消, 换底引入的 $(\log b)^p$ 为常数。令 $j = \log_b n - i$, 对非叶层求和, 就得到与 $n^k \sum_{j=1}^{\log_b n} j^p$ 同阶的结果。使用积分计算上下界可得: 当 $p > -1$ 时, 和为 $\Theta((\log_b n)^{p+1})$; 当 $p = -1$

时, 和为 $\Theta(\log(\log_b n))$; 当 $p < -1$ 时, 和为 $\Theta(1)$ 。再加上叶子的 $\Theta(n^k)$ 成本, 就得到 $a = b^k$ 对应的三种结论。

若 $a > b^k$, 则 $\log_b a > k$ 。选取 $0 < \varepsilon < \log_b a - k$ 后, 指数 $\log_b a - \varepsilon - k$ 仍为正, 因此 $(\log n)^p = O(n^{\log_b a - \varepsilon - k})$, 从而 $n^k(\log n)^p = O(n^{\log_b a - \varepsilon})$ 。这满足主定理情况 1, 得到 $T(n) = \Theta(n^{\log_b a})$ 。

若 $a < b^k$, 令 $g(n) = n^k(\log n)^p$ 。 $g(n)$ 比 $n^{\log_b a}$ 大一个多项式因子, 且

$$\frac{ag(n/b)}{g(n)} = \frac{a}{b^k} \left(1 - \frac{\log b}{\log n}\right)^p \rightarrow \frac{a}{b^k} < 1.$$

由于这个极限小于 1, 可以选择一个介于 a/b^k 与 1 之间的常数 c , 使充分大的 n 满足 $ag(n/b) \leq cg(n)$ 。因此, 以 $g(n)$ 为附加成本的递推式满足主定理情况 3, 其解为 $\Theta(g(n))$ 。 $\square$

根据这一定理, 可以直接得到例 7.6 的解。对于 $T(n) = 2T(n/2) + n \log n$, 也不难直接得到 $T(n) = \Theta(n \log^2 n)$ 。

7.2 分治法应用

在此前的基础课程中, 我们已经多次接触分而治之的思想。这一节先来回顾曾经学过的那些基础问题, 再用递推式给出分析, 然后再介绍一些新的应用。为简化表述, 本节介绍算法时统一用 O 表示时间复杂度的上界。

7.2.1 基础问题回顾

归并排序与快速排序

这应当是大部分读者首次了解分治法的例子。归并排序把数组分成两半分别排序, 长度为 1 就直接返回, 再把两个有序数组在线性时间内归并, 因此递推式为 $T(n) = 2T(n/2) + O(n)$ 。由主定理情况 2 可知总时间为 $O(n \log n)$ 。

快速排序先从数组中选出一个元素作为划分基准 (pivot), 把元素分成小于、等于和大于基准值的三组。中间一组已经处在正确的相对位置, 只需递归排序另外两组, 再依次拼接。划分过程扫描数组一次, 花费线性时间。然而, 如果 pivot 选取不当, 划分可能非常不平衡, 导致递归深度过大。最坏情况下, pivot 总是数组的最小或最大元素, 划分后一个子数组为空, 另一个子数组包含剩余的所有元素。此时递推式为 $T(n) = T(n-1) + O(n)$, 解这个递推式得到 $T(n) = O(n^2)$ 。在随机算法一讲中, 我们将通过随机选择基准值, 证明对任意固定输入都有期望 $O(n \log n)$ 的时间界。

最大子序列和问题

在最大子序列和问题中, 我们要在数组中找出和最大的一段连续元素。对数组区间 $A[\text{low}..\text{high}]$, 取中点 mid 后, 最优的子数组 $A[i..j]$ 只可能属于以下三种情况: (1) 完全在 $A[\text{low}..\text{mid}]$ 中; (2) 完全在 $A[\text{mid}+1..\text{high}]$ 中; (3) 跨越两半, 即 $\text{low} \leq i \leq \text{mid} < j \leq \text{high}$ 。

所以想法就是: (1) 分治: 将原数组平分为两半 $A[\text{low}..\text{mid}]$ 和 $A[\text{mid}+1..\text{high}]$, 然后分别对这两半求解最大子序列和; 一定不能忘记递归有 base case, 这里的 base case 就是数组只剩下一个元素, 那就什么都不用操作, 直接返回进入下一步合并阶段; (2) 合并: 首先要计算跨越 mid 和 $\text{mid}+1$ 两边的最大子序列和, 然后和左右两半的结果比较, 选择最大的作为最终结果。关键在于计算跨越两边的最大子序列和, 这其实是线性的, 为什么? 因为这里的子序列都必须跨越 mid 和 $\text{mid}+1$, 所以

最大子序列和就是最大的 $A[i, mid]$ 加上最大的 $A[mid + 1, j]$, 这个只需要对 i, j 做遍历就行, 所以是线性的。由此得到的递推公式就是很简单的 $T(n) = 2T(n/2) + O(n)$, 所以就是 $O(n \log n)$ 的复杂度。当然这个问题还有线性时间算法, 将在动态规划一讲中介绍。

逆序对计数

在数组 A 中, 若两个下标满足 $i < j$ 且 $A[i] > A[j]$, 就称 (i, j) 为一个逆序对。把数组分成左右两半后, 逆序对同样分为三类: 两个下标都在左半、都在右半, 以及一个在左半、另一个在右半。前两类递归计数, 第三类需要在合并时计算。如果使用最朴素的方法, 跨越中点的下标对最多有 $\lfloor n/2 \rfloor \lfloor n/2 \rfloor$ 个, 逐对比较仍需平方时间, 即递推式为 $T(n) = 2T(n/2) + O(n^2)$, 总时间为 $O(n^2)$, 并没有改进最原始的枚举算法的复杂度。

事实上, 我们可以在递归计数的同时, 把处理过的区间排好序。这样, 合并时就得到两个有序数组 L, R , 就能利用有序性一次数出多个逆序对。设当前尚未比较的首元素为 $L[i], R[j]$ 。如果 $L[i] \leq R[j]$, 则 $L[i]$ 不大于右边任何尚未比较的元素, 接下来继续比较 $L[i + 1]$ 即可。如果 $R[j] < L[i]$, 由于左边已经有序, $L[i], \dots$ 全都大于 $R[j]$, 它们与这个右侧元素恰好构成 $|L| - i + 1$ 个逆序对。总而言之, 可以根据这一思路在线性时间遍历两个数组, 统计出跨越中点的逆序对数。所以时间递推式与归并排序相同, 总时间为 $O(n \log n)$ 。

例如, 设左右两半排好序的数组分别为 $(2, 4, 7)$ 和 $(1, 4, 6)$ 。合并时首先判断出右边的 1 与左边三个元素都构成逆序对, 计数增加 3; 然后考虑 $(2, 4, 7)$ 和 $(4, 6)$ 计数, 发现左边的 2 和 4 不构成逆序对; 然后考虑 (7) 和 $(4, 6)$ 计数, 处理右边的 4 时计数增加 1, 然后右边的 6 再增加 1, 从而我们可以通过线性时间遍历计算出跨越中点的逆序对共有 $3 + 1 + 1 = 5$ 个。

7.2.2 最近点对问题

给定平面上的 n 个点, 最近点对问题要求我们找到欧氏距离最小的一对。最简单的想法是枚举 $\binom{n}{2}$ 个点对, 计算它们的距离, 时间复杂度为 $O(n^2)$ 。本节的目标是使用分治法得到 $O(n \log n)$ 的算法。自然地, 我们首先把点分成左右两组, 那么最优点对可能完全位于左组、完全位于右组, 也可能两组各有一个点。前两种情况递归求解即可, 要使整个算法复杂度为 $O(n \log n)$, 关键是在线性时间内找出可能更近的跨组点对。

为了实现这一目标, 我们给每个点分配唯一编号, 并分别按横坐标 x 和纵坐标 y 排序, 得到数组 P_x, P_y 。我们把 P_x 中前 $\lfloor n/2 \rfloor$ 个点分到左组, 其余点分到右组, 并用左组最后一个点的横坐标 $\bar{x}$ 画出分割线。这样, 左组点的横坐标都不大于 $\bar{x}$, 右组点的横坐标都不小于 $\bar{x}$ 。

设左右两组递归得到的最近距离分别为 δ_L, δ_R , 取 $\delta = \min(\delta_L, \delta_R) > 0$ 。如果一对跨组点的距离小于 δ , 由于分割线位于两点之间, 每个点到分割线的水平距离也必须小于 δ 。因此, 只需检查条带 $|x - \bar{x}| < \delta$ 中的点。然后将条带内的点按 y 坐标从小到大排列为 $q_1, \dots, q_s$, 依次考察每一个点。具体而言, 遍历到其中一点 q_i 时, 若能有点与它组成更近的点对, 纵坐标之差必须小于 δ , 因此这些候选点都位于图 7.3 中宽 2δ 、高 δ 的矩形内。注意高只考虑 δ 是因为每个点都考虑上方的点就足够了, 因为考虑下方的点和下方的点考虑上方的点会引入重复。

我们可以进一步将这个矩形划分为八个边长 $\delta/2$ 的正方形。这样划分的好处是什么呢? 注意到若两个同在左或右组的点若落在同一小格内, 距离至多为小格的对角线长度 $\delta/\sqrt{2} < \delta$, 与距离最小为 δ 矛盾, 因此每个小格中至多有一个点, 总共至多能容纳八个点, 其中已经包括 q_i 本身。因此, 按条带中的次序, 可能与 q_i 组成更近点对的点至多是它后面的七个点。因此在处理跨组的情况时, 我

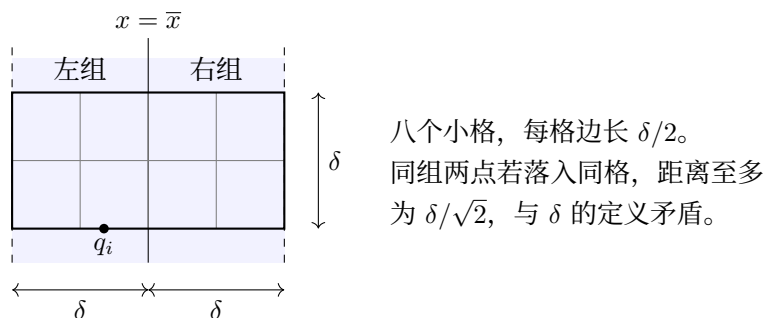


图 7.3 条带示意图。

们只需从低到高遍历条带中的所有点，每个点只需要检查它后面的七个点，计算距离并更新最小距离即可。

下面分析时间复杂度。开始时两次排序需要 $O(n \log n)$ 时间，注意排序完成后，此后的递归调用只需要利用已经排好的数组，而不需要再次排序。每次递归调用将数组分为左右平均的两部分，并且在合并时只需遍历一次条带中的点，每个点只需要常数时间计算距离并更新最小距离。因此递推式为 $T(n) = 2T(n/2) + O(n)$ ，加上预排序后，总时间仍为 $O(n \log n)$ 。

进一步减少距离计算：Basic-2

前面的算法在合并时，把条带内的所有点按纵坐标排序，让每个点与后面的至多七个点计算距离。这里面有些点对的两个点同属左组或右组，而同组的最近距离已经由递归求出，不必再检查。Basic-2 进一步减少了合并时的距离计算：每个点只需与另一组中的至多两个点比较。

先说明这两个点如何选取。沿用前面的分割线 $x = \bar{x}$ 和 $\delta = \min(\delta_L, \delta_R)$ ，我们仍然只考虑满足 $|x - \bar{x}| < \delta$ 的点。在这些点中，把原来属于左组的点按纵坐标从小到大存入数组 Y_L ，把原来属于右组的点按同样顺序存入数组 Y_R 。这两个数组可以通过扫描已有的 P_y 得到：遇到条带内的点，就按它原来的组别追加到对应数组，因此不需要重新排序。

接下来同时遍历 Y_L 和 Y_R 。用 i, j 分别记录两个数组当前遍历到的位置，并记当前找到的最小距离为 d （初始取 δ ）。只要两个数组都还没有遍历完，就执行以下步骤：

1. 比较 $Y_L[i]$ 和 $Y_R[j]$ 的纵坐标，先处理纵坐标较小的那个点；
2. 如果选中的是 $Y_L[i]$ ，就计算它到 $Y_R[j]$ 和 $Y_R[j+1]$ 的距离，然后令 $i \leftarrow i+1$ 。如果选中的是 $Y_R[j]$ ，就计算它到 $Y_L[i]$ 和 $Y_L[i+1]$ 的距离，然后令 $j \leftarrow j+1$ 。若另一数组只剩一个点，就只计算这一个距离。
3. 每算出一个小于 d 的距离，就更新 d 和保存的相应点对。

例如，假设 $\bar{x} = 0, \delta = 1$ ，条带内左组只有 $P = (-0.1, 0)$ ，右组有 $A = (0.8, 0)$ 和 $B = (0.1, 0.8)$ 。 P, A 的纵坐标相等，处理顺序不重要，可以先处理左组的 P 。这一轮算出 $|PA| = 0.9$ ， $|PB| = \sqrt{0.68} < 0.9$ ，因此 d 先由 1 更新为 0.9，再更新为 $\sqrt{0.68}$ ，保存的点对为 (P, B) ，左组遍历完毕，扫描结束。事实上，这个例子也说明了只比较另一数组的第一个点还不够。

为什么比较两个点就够了？我们已经知道同组内任意两点的距离都至少为 δ ，事实上这个条件限制了另一组中候选点的位置。下面证明所需的结论。

证明. 假设当前处理的是左组的点 P ，右组的情况对称故不赘述。将右组候选点中按纵坐标排序的第一个点记为 A 。如果 A 已经离 P 最近，结论就成立；否则，取一个离 P 最近的点 C ，于是 $|PC| < |PA|$

且 $|PC| < \delta$ 。我们将证明 C 必须排在第二个位置。假如它排在第三个或更后面, A, C 之间就至少还有一个点 B , 下面说明这会使 A, B, C 中的两个点距离小于 δ , 与它们同属右组矛盾。

平移坐标, 使分割线成为 $x = 0$, 并使 P 的纵坐标为 0。记 $P = (-h, 0)$, $A = (a, v)$, $C = (c, w)$, 其中 $h \geq 0$, $0 \leq a, c \leq \delta$, $0 \leq v \leq w$ 。 C 的纵坐标不小于 A , 却比 A 更靠近 P , 所以 C 的横坐标必须更小, 即 $c < a$ 。把 A, C 的水平距离和竖直距离分别记为 $X = a - c > 0$, $D = w - v \geq 0$ 。先由 C 更靠近 P 这一条件, 估计 D 的大小。如果 $h + a \leq \delta$, 将 $|PC|^2 < |PA|^2$ 按坐标展开, 得到

$$D^2 \leq w^2 - v^2 < (h + a)^2 - (h + c)^2 = 2(h + a)X - X^2 \leq 2\delta X - X^2.$$

如果 $h + a > \delta$, 则 $h + c = h + a - X > \delta - X \geq 0$ 。此时利用 $|PC| < \delta$, 得到

$$D^2 \leq w^2 < \delta^2 - (h + c)^2 < \delta^2 - (\delta - X)^2 = 2\delta X - X^2.$$

因此, 两种情况都满足 $(\delta - X)^2 + D^2 < \delta^2$ 。也就是说, 只要两点的水平距离不超过 $\delta - X$ 、竖直距离不超过 D , 它们的距离就小于 δ 。

现在考察排在 A, C 之间的点 $B = (z, t)$ 。因为三个点按纵坐标排序, 所以 $v \leq t \leq w$; 又因为 B 在右侧条带内, 所以 $0 \leq z \leq \delta$ 。根据 B 的横坐标, 分以下三种情况:

- 若 $z \leq c$, 则 B, C 的水平距离至多为 c 。注意 $c = a - X \leq \delta - X$, 且 B, C 竖直距离至多为 D , 可得 $|BC| < \delta$ 。
- 若 $z \geq a$, 则 A, B 的水平距离至多为 $\delta - a \leq \delta - X$, 竖直距离至多为 D , 所以 $|AB| < \delta$ 。
- 若 $c < z < a$, 则 B 的两个坐标分别位于 A, C 的相应坐标之间。利用两个非负数的平方和不超过它们的和的平方, 得到

$$\begin{aligned} |AB|^2 + |BC|^2 &= (a - z)^2 + (z - c)^2 + (t - v)^2 + (w - t)^2 \\ &\leq (a - c)^2 + (w - v)^2 = X^2 + D^2 < 2\delta^2. \end{aligned}$$

最后一步使用了 $X \leq \delta$ 和 $D \leq w < \delta$, 其中 $w < \delta$ 来自 $|PC| < \delta$ 。因此 $|AB|$ 和 $|BC|$ 不可能都不小于 δ 。

三种情况都出现了距离小于 δ 的同组点对, 产生矛盾。因此, A, C 之间不可能还有其他点, C 必须是第二个点。□

总结而言, Basic-2 保留了原算法的 $O(n \log n)$ 总时间上界, 但把合并时每个点至多七次距离计算减少到了至多两次。Pereira 与 Lobo 的论文给出了这一算法及其对更一般距离度量的证明, 可见本讲参考资料。

7.2.3 从整数乘法到矩阵乘法

提到分治法, 不得不提的就是 Strassen 矩阵乘法。这是一个看起来有点魔法, 能将矩阵乘法从最简单的三次方算法复杂度下降的方法。为了使这一方法看起来不那么魔法, 可以先介绍 Karatsuba 整数乘法算法作为基础。

考虑两个长度为 n 的整数做乘法 (长度不同则短的高位补 0 补齐长度即可), 如果不做优化, 用传统的竖式乘法显然复杂度为 $O(n^2)$ 。如果用分治法, 想法很简单, 就是将两个整数分为两半分别

计算乘法，然后组合起来即可。将长度平分，可以设第一个乘数为 $a = 10^{n/2}a_1 + a_2$ ，第二个乘数为 $b = 10^{n/2}b_1 + b_2$ ，那么二者相乘的结果就是

$$ab = 10^n a_1 b_1 + 10^{n/2}(a_1 b_2 + a_2 b_1) + a_2 b_2.$$

因此在分治阶段需要计算 $a_1 b_1, a_1 b_2, a_2 b_1, a_2 b_2$ 四个长度为 $n/2$ 的乘法，合并阶段只需要简单移位和加法就行，复杂度是线性的，所以整体时间复杂度递推公式为 $T(n) = 4T(n/2) + O(n)$ 。尴尬的事情出现了，根据主定理这个时间复杂度是 $O(n^2)$ ，似乎优化失败了。其实关键就在于这里分治要计算四个乘法，如果能计算更少的乘法呢？注意到

$$a_1 b_2 + a_2 b_1 = (a_1 + a_2)(b_1 + b_2) - a_1 b_1 - a_2 b_2,$$

其中 $a_1 b_1$ 和 $a_2 b_2$ 本来就需要计算。因此，我们只需递归求出 $a_1 b_1$ 、 $a_2 b_2$ 和 $(a_1 + a_2)(b_1 + b_2)$ 三个乘积，再用后者减去前两个乘积，就得到交叉项 $a_1 b_2 + a_2 b_1$ 。从而得到递推式

$$T(n) = 3T(n/2) + O(n),$$

由主定理得到 $O(n^{\log_2 3})$ ，比平方时间更低。当然有一个细节需要注意，就是计算加法的过程中可能会产生进位，因此递推式严谨而言是 $T(n) = 3T(n/2 + 1) + O(n)$ ，但这并不影响最终的复杂度，这一点可以通过习题 7.2 理解。

有了上面的基础，下面来看矩阵乘法。分治的想法显然是使用分块矩阵，考虑简单的情况，即矩阵是方阵的情况，先将阶数补到不小于 n 的最小 2 的幂 N （新增行列填零），以下用 n 表示补齐后的阶数，将两个矩阵进行如下分块：

$$A = \begin{pmatrix} A_1 & A_2 \\ A_3 & A_4 \end{pmatrix}, \quad B = \begin{pmatrix} B_1 & B_2 \\ B_3 & B_4 \end{pmatrix},$$

设结果矩阵

$$C = \begin{pmatrix} C_1 & C_2 \\ C_3 & C_4 \end{pmatrix},$$

则有 $C_1 = A_1 B_1 + A_2 B_3$, $C_2 = A_1 B_2 + A_2 B_4$, $C_3 = A_3 B_1 + A_4 B_3$, $C_4 = A_3 B_2 + A_4 B_4$ 。于是分治需要计算 8 个阶数为 $n/2$ 的矩阵乘法，然后用平方时间做矩阵加法即可，即时间复杂度递推公式为 $T(n) = 8T(n/2) + O(n^2)$ 。很遗憾，时间复杂度仍然是 $O(n^3)$ 。所以类似于 Karatsuba 整数乘法，我们希望用更少的乘法来表达出 8 个乘法。天才的 Strassen 在 1969 年给出了这一算法，他构造了如下七个矩阵

$$M_1 = (A_1 + A_4)(B_1 + B_4)$$

$$M_2 = (A_3 + A_4)B_1$$

$$M_3 = A_1(B_2 - B_4)$$

$$M_4 = A_4(B_3 - B_1)$$

$$M_5 = (A_1 + A_2)B_4$$

$$M_6 = (A_3 - A_1)(B_1 + B_2)$$

$$M_7 = (A_2 - A_4)(B_3 + B_4)$$

通过组合上述 7 个新矩阵得到：

$$C_1 = M_1 + M_4 - M_5 + M_7, C_2 = M_3 + M_5, C_3 = M_2 + M_4, C_4 = M_1 - M_2 + M_3 + M_6.$$

于是只需要 $O(n^2)$ 次标量运算计算这些矩阵的加法和减法，然后分治只需要计算 7 个乘法即可，因此时间复杂度递推公式为

$$T(n) = 7T(n/2) + O(n^2),$$

因此复杂度变为了 $O(n^{\log_2 7})$ ，这相比三次方有了一定改进。当然矩阵乘法计算复杂度的优化仍然是一个活跃的方向，感兴趣的读者可以查阅相关文献，甚至你也可以尝试使用前沿大模型智能体来帮助你寻找更优的矩阵乘法算法。

7.2.4 确定性线性时间选择

给定一个未排序数组，选择问题要求找出其中第 k 小的元素。最直接的想法当然是先排序再取第 k 项，但事实上如果只为找一个排名，不一定需要排出全部元素的顺序。

为了能在更快的时间（线性时间）内解决问题，我们沿用快速排序的三路划分：选定基准值 v ，把小于、等于和大于 v 的元素分别放入 L, E, G 。 L 中的元素占据前 $|L|$ 个排名，接下来 $|E|$ 个排名的值全是 v ，其余排名落在 G 。因此，若 $k \leq |L|$ ，只需在 L 中找第 k 小；若 $|L| < k \leq |L| + |E|$ ，直接返回 v ；否则在 G 中找第 $k - |L| - |E|$ 小。然而，如果基准值选得不好，仍然可能花费平方时间。例如每次基准值都选到最小值，而目标是最大值时，问题规模每次只减少 1。我们希望选出一个基准值，保证无论答案在哪一侧，下一次递归都能排除固定比例的元素。

为此，把数组每五个元素分成一组，在各组内排序并取中位数，再递归选择这些中位数的中位数作为 v （如图 7.4）。由于组内至多五个元素，排序只需常数时间；中位数数组则只有 $\lceil n/5 \rceil$ 项，因此找 v 本身也是一个规模更小的选择问题。

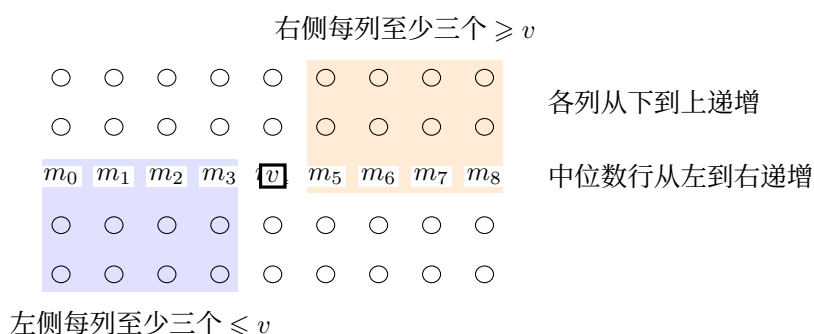


图 7.4 九个完整五元组，每列对应一组，图中按各组中位数从小到大排列各列。

在选择了 v 后需要计算至少有多少元素不小于 v 。设共有 $g = \lceil n/5 \rceil$ 组。由于 v 是中位数数组的中位数，其中至少 $g/2$ 项不小于 v 。如果某个完整五元组的中位数不小于 v ，该组的中位数以及排在它后面的两个元素就都不小于 v ，至少贡献三个元素。除去可能不足五项的一组 and 提供所选中位数 v 的一组，仍然至少有 $3(g/2 - 2) \geq 3n/10 - 6$ 个元素不小于 v 。按同样的方法，从中位数不大于 v 的组开始数，也能得到至少 $3n/10 - 6$ 个不大于 v 的元素。因此，严格小于 v 的组 L 至多有 $7n/10 + 6$ 项，严格大于 v 的组 G 也有相同上界。

现在把选基准值和三路划分结合起来，得到完整的选择算法。记 $\text{Select}(A, k)$ 为在数组 A 中寻找第 k 小元素的过程，一次调用按以下顺序进行：

1. 如果数组足够短（元素个数不超过 140），就直接排序并返回第 k 项，常数 140 的选择将在复杂度分析中说明。
2. 否则，每五个元素分成一组，将各组的中位数收集到数组 M 中，再调用 $\text{Select}(M, \lceil |M|/2 \rceil)$ ，得到基准值 v 。
3. 用刚得到的 v 扫描原数组 A ，把每个元素放入 L, E, G 中对应的一组。
4. 根据 k 判断答案属于哪一组。如果属于 E ，直接返回 v ；如果属于 L ，调用 $\text{Select}(L, k)$ ；如果属于 G ，调用 $\text{Select}(G, k - |L| - |E|)$ 。

因此，一次 Select 调用要解决两个更小的选择问题：先从 $\lceil n/5 \rceil$ 个组中位数中选出 v ，再到 L 或 G 中寻找目标元素，后一次的规模至多为 $\lfloor 7n/10 + 6 \rfloor$ 。除此之外，组内排序、收集中位数和三路划分合计都只需 $O(n)$ 时间。因此有递推关系

$$T(n) \leq T(\lceil n/5 \rceil) + T(\lfloor 7n/10 + 6 \rfloor) + cn.$$

两个递归调用的规模不同，这里用代入法证明线性上界。假设更小规模已经满足 $T(m) \leq dm$ ，代入右边得到 $d(n/5 + 1) + d(7n/10 + 6) + cn = dn - (d/10 - c)n + 7d$ 。要使结果不超过 dn ，只需让后面的 $-(d/10 - c)n + 7d$ 不大于 0。先取 $d \geq 20c$ ，便有 $d/10 - c \geq d/20$ ；再利用 $n > 140$ ，得到 $-(d/10 - c)n + 7d \leq -dn/20 + 7d \leq 0$ 。最后把 d 取得足够大，使所有 $n \leq 140$ 的初值也满足 $T(n) \leq dn$ ，归纳证明就完成了。当然根据上述证明可知，常数 140 的选取并不唯一，只需要是一个足够大的常数即可。

7.3 补充材料

7.3.1 不均匀分治与 Akra–Bazzi 公式

在此前的讨论中，我们用递归树求出了 $T(n) = T(n/3) + T(2n/3) + cn$ 对应的界。这个递推式有两个不同的缩小比例，无法写成主定理要求的 $aT(n/b) + f(n)$ ，需要使用接下来介绍的 Akra–Bazzi 公式处理。值得指出的是，本节给出的 Akra–Bazzi 公式是最基础的版本，推广的版本可参阅参考资料中 Leighton 的讲义。

设 $T(x) = \sum_{i=1}^q a_i T(b_i x) + g(x)$ ，即按子问题规模把递归调用分成 q 类，第 i 类的规模为原来的 b_i 倍， a_i 是这一类递归调用的次数， $g(x)$ 是本次调用在所有递归调用之外所需的时间。当然，公式本身允许 a_i 为任意正常数，不必是整数，因此也不一定要解释为递归调用的次数。附加成本 g 还需要满足一个条件：当输入规模在固定比例范围内变化时，它的值至多改变常数倍。具体地，对充分大的 x 、每个 i 以及 $b_i x \leq u \leq x$ ，存在统一的正常数 c_1, c_2 ，使得

$$c_1 g(x) \leq g(u) \leq c_2 g(x). \quad (7.2)$$

例如，若 $g(x) = x^k$ ，则 $g(u)/g(x) = (u/x)^k$ ，而 u/x 介于固定的 b_i 与 1 之间，所以满足这个条件。对充分大的输入， $x^k (\log x)^p$ 也满足它，其中 k, p 都是固定实数。

定理 7.9 (Akra-Bazzi 公式). 设 q 为正整数, $a_i > 0$, $0 < b_i < 1$ 都是固定常数. 取足够大的常数 x_0 , 使得 $x > x_0$ 时 $1 \leq b_i x \leq x - 1$, 并且式 (7.2) 成立. 对 $x \geq 1$, 设

$$T(x) = \begin{cases} \Theta(1), & 1 \leq x \leq x_0, \\ \sum_{i=1}^q a_i T(b_i x) + g(x), & x > x_0, \end{cases}$$

其中 g 非负且在每个有限区间上可积. 令 s 为方程

$$\sum_{i=1}^q a_i b_i^s = 1 \quad (7.3)$$

的解, 则

$$T(x) = \Theta\left(x^s \left(1 + \int_1^x \frac{g(u)}{u^{s+1}} du\right)\right). \quad (7.4)$$

使用公式时, 先由式 (7.3) 求出 s , 再把附加成本 g 代入积分 (式 (7.4)). 式 (7.3) 恰有一个实数解: 由于 $0 < b_i < 1$, 每一项 $a_i b_i^s$ 都关于 s 连续且严格递减, 且和式在 $s \rightarrow -\infty$ 时趋于无穷, 在 $s \rightarrow +\infty$ 时趋于 0, 因此只会有一次取到 1。

求 s 的方程来自递推式本身. 先暂时去掉附加成本 $g(x)$, 只看 $T(x) = \sum_{i=1}^q a_i T(b_i x)$, 并尝试用幂函数 $T(x) = x^s$ 满足这个等式. 代入后得到

$$x^s = \sum_{i=1}^q a_i (b_i x)^s = x^s \sum_{i=1}^q a_i b_i^s.$$

两边除以 x^s , 就得到式 (7.3). 因此, s 由递归调用的系数 a_i 和输入缩小的比例 b_i 决定. 例如, 对于主定理中的递归项 $aT(x/b)$, 上述方程就是 $a(1/b)^s = 1$, 解得 $s = \log_b a$.

在式 (7.4) 中, 如果 $g = 0$, 积分也为 0, 结论便是 $T(x) = \Theta(x^s)$. 当 a_i 表示递归调用的次数时, 此时只计算叶子处的终止成本, 所以 x^s 反映了叶子数量的增长阶. 加回 g 后, 还要计入各次非叶调用自身的工作; 公式中的 $x^s \int_1^x g(u)/u^{s+1} du$ 就用于计算这部分成本对总时间的贡献。

例题 7.10. 考虑 $T(x) = T(x/3) + T(2x/3) + cx$, 其中 $c > 0$, 常数规模时递归停止. 用 Akra-Bazzi 公式求 $T(x)$ 的渐近紧界. 如果将附加成本 cx 改为 x^2 , 结果如何变化?

解答. 由 $(1/3)^s + (2/3)^s = 1$ 可得 $s = 1$. 于是

$$\int_1^x \frac{g(u)}{u^{s+1}} du = \int_1^x \frac{cu}{u^2} du = c \ln x,$$

代入式 (7.4), 得到 $T(x) = \Theta(x(1 + c \ln x)) = \Theta(x \log x)$, 因此结果与前面的递归树分析一致。

如果只把附加成本改为 x^2 , 那么 s 仍为 1, 但积分变成 $\int_1^x \frac{u^2}{u^2} du = x - 1$, 此时得到 $T(x) = \Theta(x(1 + x - 1)) = \Theta(x^2)$. $\square$

Akra-Bazzi 公式的证明. 把式 (7.4) 中的函数记为

$$H(x) = x^s \left(1 + \int_1^x \frac{g(u)}{u^{s+1}} du\right).$$

我们将证明： $H(x)$ 的某个正常数倍是 $T(x)$ 的下界，另一个正常数倍是上界。为此，先计算 $H(x)$ 与其递归子项之和的差。利用式 (7.3)，得到

$$H(x) - \sum_i a_i H(b_i x) = x^s \sum_i a_i b_i^s \int_{b_i x}^x \frac{g(u)}{u^{s+1}} du.$$

积分区间中， $g(u)$ 与 $g(x)$ 只相差常数倍， u 与 x 也只相差常数倍，而区间长度为 $(1 - b_i)x$ 。因此每个积分与 $x \cdot \frac{g(x)}{x^{s+1}} = \frac{g(x)}{x^s}$ 只相差常数倍。因此存在正常数 A, B ，使得

$$Ag(x) \leq H(x) - \sum_i a_i H(b_i x) \leq Bg(x). \quad (7.5)$$

取足够小的正常数 λ ，使所有满足 $1 \leq x \leq x_0$ 的输入都有 $T(x) \geq \lambda H(x)$ ，同时满足 $\lambda B \leq 1$ 。由式 (7.5) 的右侧不等式可得 $\lambda H(x) \leq \sum_i a_i \lambda H(b_i x) + g(x)$ 。假设对子问题已有 $T(b_i x) \geq \lambda H(b_i x)$ ，由于 $a_i > 0$ ，就有 $\lambda H(x) \leq \sum_i a_i \lambda H(b_i x) + g(x) \leq \sum_i a_i T(b_i x) + g(x) = T(x)$ 。由归纳法即证明了下界 $T(x) \geq \lambda H(x)$ 。上界也按同样的思路证明。取足够大的正常数 Λ ，使所有满足 $1 \leq x \leq x_0$ 的输入都有 $T(x) \leq \Lambda H(x)$ ，同时满足 $\Lambda A \geq 1$ 。由式 (7.5) 的左侧不等式可得 $\sum_i a_i \Lambda H(b_i x) + g(x) \leq \Lambda H(x)$ 。假设各个子问题满足 $T(b_i x) \leq \Lambda H(b_i x)$ ，代入原递推式便得到 $T(x) = \sum_i a_i T(b_i x) + g(x) \leq \sum_i a_i \Lambda H(b_i x) + g(x) \leq \Lambda H(x)$ 。综合上述讨论有 $\lambda H(x) \leq T(x) \leq \Lambda H(x)$ ，即 $T(x) = \Theta(H(x))$ 。□

7.3.2 快速傅里叶变换 (FFT)

前面介绍了利用分治加速整数乘法的方法。对于多项式乘法，也可以把计算分成较小的子问题，再合并结果。**快速傅里叶变换** (Fast Fourier Transform, FFT) 正是这样一种分治算法：它能快速求出多项式在一组特定点上的值，然后将两个多项式在相同位置的值分别相乘，再由这些乘积恢复系数，就能求出多项式的乘积。

系数表示与点值表示

设

$$A(x) = \sum_{i=0}^{u-1} a_i x^i, \quad B(x) = \sum_{j=0}^{v-1} b_j x^j, \quad u, v \geq 1.$$

展开乘积 $C(x) = A(x)B(x)$ ，只有下标之和为 s 的两项相乘才会产生 x^s ，因此它的系数为

$$c_s = \sum_{\substack{0 \leq i < u, 0 \leq j < v \\ i+j=s}} a_i b_j, \quad 0 \leq s \leq u+v-2.$$

这样得到的系数数组 c 称为 a, b 的**卷积** (convolution)。若两个输入数组都长 n ，直接展开需要计算 n^2 个乘积，下面利用 FFT 将算术运算次数降到 $O(n \log n)$ 。

求乘积的系数需要许多乘加运算，但求某一点 z 处的值就简单得多：只要分别知道 $A(z)$ 、 $B(z)$ ，相乘就是 $C(z)$ 。这启发我们先计算两个多项式在同一组点上的值，再逐点相乘，最后由乘积的点值恢复系数。如果两次转换都能做得足够快，多项式乘法也就能加速。这里用到的基本数学原理是：次数小于 n 的多项式，在 n 个不同点上的取值能够唯一确定其系数。否则，两个不同的多项式在这些点上的值相同，它们相减就得到一个次数至多为 $n-1$ 、却有 n 个不同根的非零多项式，产生矛盾。

由于 $C(x)$ 的次数至多为 $u+v-2$ ，要由点值确定它的全部系数，就需要选取至少 $u+v-1$ 个不同的点。把选取的点数记为 n ，便要求 $n \geq u+v-1$ 。为了接下来统一处理，将 A, B 的系数数组都在高次项补零至长度 n 。由于补上的只是高次项的零系数，多项式本身没有改变。

不过，改用点值表示还没有解决计算量的问题。如果对每个点都单独求值，长度为 n 的系数数组需要逐项参与计算，求出 n 个点值仍需 $\Theta(n^2)$ 次运算。接下来要解决的就是怎样更快地求出这些点值。FFT 会专门选择一组适合分治的点，使一次递归计算的结果可以同时用于多个点的求值。

利用单位根进行分治

要找到适合分治的求值点，可以利用复数中的单位根。先回顾两个基本事实： $i^2 = -1$ ，以及

$$(\cos \alpha + i \sin \alpha)(\cos \beta + i \sin \beta) = \cos(\alpha + \beta) + i \sin(\alpha + \beta).$$

后一个等式说明单位圆上复数相乘就是角度相加。令

$$\omega_n = \cos \frac{2\pi}{n} + i \sin \frac{2\pi}{n}.$$

则 $1, \omega_n, \dots, \omega_n^{n-1}$ 是 n 个互异的 n 次单位根：它们的 n 次方都为 1，对应角度 $0, 2\pi/n, \dots, 2\pi(n-1)/n$ 。使 $\omega_n^k = 1$ 的最小正整数为 n ，也称 ω_n 为 n 次本原单位根，也说它的阶为 n 。对任意整数 k ， $\omega_n^k = 1$ 当且仅当 n 整除 k 。取 $n = 2^k$ 。当 $n \geq 2$ 时，有 $\omega_n^{n/2} = -1$ ， $\omega_n^2 = \omega_{n/2}$ 。因此，第 j 个单位根与第 $j + n/2$ 个单位根互为相反数，平方后便相同。而平方得到的不同值，恰好是全部 $n/2$ 次单位根。

我们希望利用平方后求值点重合的性质，让 $A(z)$ 和 $A(-z)$ 的计算共用一部分结果。为此，可以将 $A(x)$ 的偶数次项和奇数次项分开：

$$A(x) = E(x^2) + xO(x^2), \quad E(y) = \sum_{j=0}^{n/2-1} a_{2j}y^j, \quad O(y) = \sum_{j=0}^{n/2-1} a_{2j+1}y^j.$$

$A(z)$ 和 $A(-z)$ 分别等于 $E(z^2) + zO(z^2)$ 和 $E(z^2) - zO(z^2)$ ，用到的都是同一对 $E(z^2)$ 、 $O(z^2)$ 。 n 个单位根平方后只有 $n/2$ 个不同的值，因此只需分别求出 E 、 O 在这 $n/2$ 个点上的值，就能算出 A 在全部 n 个点上的值。而 E 、 O 也各自只有 $n/2$ 个系数，求值点恰好是全部 $n/2$ 次单位根，所以这两个子问题都可以继续用相同的方法求解。设已经递归求得 $e_j = E(\omega_{n/2}^j)$ ， $o_j = O(\omega_{n/2}^j)$ ，其中 $0 \leq j < n/2$ ，就得到

$$A(\omega_n^j) = e_j + \omega_n^j o_j, \quad A(\omega_n^{j+n/2}) = e_j - \omega_n^j o_j.$$

由一对 e_j, o_j 算出这两个点值，只需一次乘法、一次加法和一次减法，这样的合并称为**蝶形运算**。对 $j = 0, \dots, n/2 - 1$ 分别进行这一步，就能从两个子问题的结果得到全部 n 个点值。

DFT 与 FFT 算法

把 $A(x)$ 在 $1, \omega_n, \dots, \omega_n^{n-1}$ 处的值依次记为 $\hat{a}_0, \dots, \hat{a}_{n-1}$ 。从系数数组 a 计算这个点值数组的过程，称为**离散傅里叶变换** (Discrete Fourier Transform, DFT)，即 $\hat{a}_j = \sum_{k=0}^{n-1} a_k \omega_n^{jk}$ ，其中 $0 \leq j < n$ 。直接按定义计算需要 $O(n^2)$ 次运算，前面的分治方法能够快速完成这一计算，就是 FFT。下面将它写成递归算法：

算法 7.1 FFT

输入： $a[0..n-1]$ ， n 是正的二的幂； ω 是 n 次本原单位根

输出： 返回 $y_j = \sum_{k=0}^{n-1} a_k \omega^{jk}$ ，按 $j = 0, \dots, n-1$ 排列

1: **function** FFT(a, ω)

2: $n \leftarrow |a|$

```

3:   if  $n = 1$  then
4:     return  $[a_0]$ 
5:   end if
6:    $e \leftarrow \text{FFT}([a_0, a_2, \dots, a_{n-2}], \omega^2)$ 
7:    $o \leftarrow \text{FFT}([a_1, a_3, \dots, a_{n-1}], \omega^2)$ 
8:   分配  $y[0..n-1]$ ;  $w \leftarrow 1$ 
9:   for  $j = 0, \dots, n/2 - 1$  do
10:     $z \leftarrow w o_j$ 
11:     $y_j \leftarrow e_j + z$ ;  $y_{j+n/2} \leftarrow e_j - z$ 
12:     $w \leftarrow w \omega$ 
13:  end for
14:  return  $y$ 
15: end function

```

一次长度为 n 的调用会递归处理两个长度为 $n/2$ 的数组。除此之外，拆分系数和合并点值都只需遍历数组，所以其余工作需要 $O(n)$ 次运算。于是 $T(n) = 2T(n/2) + O(n)$, $T(1) = O(1)$ 。由主定理可得， $n \geq 2$ 时，FFT 需要 $O(n \log n)$ 次算术运算。

逆变换：从点值恢复系数

现在已经能从系数求出点值，但多项式乘法最终需要的是系数，所以还要解决反过来的问题：已知全部 $\hat{a}_j$ ，怎样求回 $a_0, \dots, a_{n-1}$ ？每个点值 $\hat{a}_j = \sum_{k=0}^{n-1} a_k \omega_n^{jk}$ 都包含所有系数。要从中取出某个 a_ℓ ，我们希望将这些等式组合起来，使其余系数的贡献相互抵消。下面这个单位根的求和性质正好可以做到这一点：对任意整数 d ,

$$\sum_{j=0}^{n-1} \omega_n^{jd} = \begin{cases} n, & n \mid d, \\ 0, & n \nmid d. \end{cases} \quad (7.6)$$

若 $n \mid d$ ，则 $\omega_n^d = 1$ ，求和中的 n 项都为 1，结果就是 n 。否则，令 $r = \omega_n^d$ ，则 $r \neq 1$ ，但 $r^n = 1$ 。利用等比数列求和公式，得到

$$1 + r + \dots + r^{n-1} = \frac{1 - r^n}{1 - r} = 0.$$

现在固定要恢复的系数下标 ℓ 。将第 j 个点值乘以 $\omega_n^{-j\ell}$ 后，其中 a_k 对应的因子就变成 $\omega_n^{j(k-\ell)}$ 。再对 j 求和，根据式 (7.6)， $k = \ell$ 时留下 $n a_\ell$ ，其余项都抵消为 0。因此，最后除以 n 就能得到 a_ℓ ：

$$\frac{1}{n} \sum_{j=0}^{n-1} \hat{a}_j \omega_n^{-j\ell} = \frac{1}{n} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} a_k \omega_n^{j(k-\ell)} = \sum_{k=0}^{n-1} a_k \left(\frac{1}{n} \sum_{j=0}^{n-1} \omega_n^{j(k-\ell)} \right) = a_\ell.$$

这个恢复过程也能用 FFT 加速。把 $\hat{a}$ 作为输入数组，将单位根换成 ω_n^{-1} ，FFT 返回的第 ℓ 项就是 $\sum_{j=0}^{n-1} \hat{a}_j \omega_n^{-j\ell}$ 。将返回数组的每项除以 n ，便得到全部系数。这个过程称为**逆变换**，只需调用一次 FFT，再做 n 次除法。

现在可以把多项式乘法的步骤合在一起了。先将两个系数数组补零到足够长（不小于 $u + v - 1$ 的最小二的幂），分别用 FFT 求出同一组点上的值，再将对应的点值相乘，得到乘积多项式的点值。最后做逆变换，就能求出乘积的系数。

算法 7.2 Convolve: 由两次正变换和一次逆变换求卷积

输入: 系数数组 $a[0..u-1], b[0..v-1]$; 空数组表示零多项式

```

1: if  $u = 0$  或  $v = 0$  then
2:   return 空数组
3: end if
4:  $L \leftarrow u + v - 1; n \leftarrow 1$ 
5: while  $n < L$  do
6:    $n \leftarrow 2n$ 
7: end while
8: 把  $a, b$  各复制并补零至长度  $n$ ;  $\omega \leftarrow \omega_n$ 
9:  $U \leftarrow \text{FFT}(a, \omega); V \leftarrow \text{FFT}(b, \omega)$ 
10: 对每个  $j = 0, \dots, n-1$ , 计算  $Z_j \leftarrow U_j V_j$ 
11:  $c \leftarrow \text{FFT}(Z, \omega^{-1})$ 
12: for  $s = 0, \dots, L-1$  do
13:    $c_s \leftarrow c_s / n$ 
14: end for
15: return  $c[0..L-1]$ 

```

三次 FFT 共需 $O(n \log n)$ 次运算, 补零、逐点相乘和除以 n 另需 $O(n)$ 次运算。由于 n 是不小于 L 的最小二的幂, 有 $L \leq n < 2L$, 因此总算术运算次数为 $O((u+v) \log(u+v))$ 。

浮点误差与数论变换 (NTT)

前面的推导假设复数运算是精确的, 但程序通常用浮点数保存复数, 只能近似表示单位根, 后续计算也会积累舍入误差。若乘积的系数应为整数, 可以将计算结果的实部四舍五入, 但要保证恢复正确, 每一项实部与真实系数的误差都必须小于 $1/2$ 。

对于只要求系数除以某个素数 p 的余数的问题, 可以改用整数运算: 我们要找到一个整数 ω , 在模 p 的意义下起到本原单位根的作用, 即

$$\omega^n \equiv 1 \pmod{p}, \quad \omega^t \not\equiv 1 \pmod{p} \quad (1 \leq t < n).$$

这时称 ω 在模 p 下的阶为 n 。用这样的 ω 代替复数单位根, 并将计算改为模 p 运算, 得到的变换称为**数论变换** (Number-Theoretic Transform, NTT)。

变换长度 n 和素数 p 不能随意选择。下面证明, 存在阶为 n 的 ω 时, 必有 $n \mid (p-1)$ 。由于 p 是素数, 非零余数 $1, 2, \dots, p-1$ 分别乘以 ω 后, 仍然非零且互不相同, 所以它们只是换了排列顺序。将这些数相乘, 得到 $\omega^{p-1}(1 \cdot 2 \cdots (p-1)) \equiv 1 \cdot 2 \cdots (p-1) \pmod{p}$ 。乘积中的每个因子都不被 p 整除, 可以在同余式两边约去, 从而得到 $\omega^{p-1} \equiv 1 \pmod{p}$ 。再将 $p-1$ 写成 $qn+r$, 其中 $0 \leq r < n$, 便有 $1 \equiv \omega^{p-1} = (\omega^n)^q \omega^r \equiv \omega^r \pmod{p}$ 。由于 n 是使 ω 的幂同余于 1 的最小正指数, 只能有 $r = 0$, 所以 n 整除 $p-1$ 。

下面检查分治公式是否仍然成立。仍取 n 为二的幂, 并设 $n \geq 2$ 。首先, $(\omega^2)^{n/2} \equiv 1 \pmod{p}$, 故 ω^2 的阶恰为 $n/2$, 否则更小的正指数会使 ω 在不足 n 次幂时就同余于 1。因此两个递归调用仍可进行。其次, 令 $z = \omega^{n/2}$, 则 $z^2 \equiv 1 \pmod{p}$, 即 $(z-1)(z+1) \equiv 0 \pmod{p}$ 。素数 p 必须整除其

中一个因子, 而 $z \not\equiv 1 \pmod{p}$, 因此 $z \equiv -1 \pmod{p}$ 。这说明求值点仍能两两配成相反数, 合并时仍使用 $e_j + \omega^j o_j$ 和 $e_j - \omega^j o_j$, 只是都要对 p 取余。

逆变换也能沿用前面的推导。在模素数 p 的运算中, 任意非零余数 a 都有**模逆元** a^{-1} , 满足 $aa^{-1} \equiv 1 \pmod{p}$, 则原来需要除以 a 的地方, 改为乘以这个逆元即可。若 n 不整除 d , 则 $1 - \omega^d$ 非零, 所以等比求和公式中的这一因子仍能约去, 式 (7.6) 在模 p 下仍成立。又因为 $n \mid (p-1)$, 有 $1 \leq n < p$, 故 n 也有模逆元。因此, 逆变换仍是以 ω^{-1} 调用同一个算法, 再将每项乘以 n^{-1} , 所有计算均对 p 取余。

例如, 用 NTT 计算 $A(x) = 1 + 2x$ 与 $B(x) = 3 + 4x$ 的乘积。乘积至多有三个系数, 因此将输入数组补成 $(1, 2, 0, 0)$ 和 $(3, 4, 0, 0)$, 取变换长度 $n = 4$, 模数 $p = 17$ 。选择 $\omega = 4$, 因为 $4^2 \equiv -1 \pmod{17}$ 、 $4^4 \equiv 1 \pmod{17}$, 所以 4 在模 17 下的阶恰为 4。

求值点依次为 1, 4, 16, 13。分别代入 A 、 B 并对 17 取余, 得到点值数组 $(3, 9, 16, 10)$ 和 $(7, 2, 16, 4)$; 对应相乘后再取余, 得到乘积的点值数组 $(4, 1, 1, 6)$ 。由于 $4 \cdot 13 \equiv 1 \pmod{17}$, 逆变换使用的单位根为 13, 长度 $n = 4$ 的模逆元也为 13。以 13 为单位根对乘积点值做 NTT, 得到 $(12, 6, 15, 0)$, 再将每项乘以 13 并取余, 就得到 $(3, 10, 8, 0)$ 。因此乘积为 $3 + 10x + 8x^2$, 与直接展开一致。这里能直接恢复整数系数, 是因为真实系数都在 0 到 16 之间。一般情况下, NTT 只能求出系数除以 17 的余数, 例如系数 3 和 20 都会得到余数 3。

7.4 习题

习题 7.1 (基础递推式分析). 判断下列结论是否正确, 并说明理由:

1. 若 $T(n) = T(n/5) + T(7n/10) + O(n)$, 则 $T(n) = O(n)$ 。如果把两个比例改为 $1/6$ 和 $7/9$, 结论是否仍成立?
2. 每次递归产生七个规模为 $n/3$ 的子问题, 且附加成本为 $O(n^2)$, 则总成本一定为 $\Theta(n^2)$ 。
3. 若 $T(n) = 2T(n/2) + n/(\log n)^2$, 则 $T(n) = \Theta(n)$ 。

习题 7.2 (带常数偏移的递推式). 设 n 为正整数, 函数 $T(n)$ 满足

$$T(n) = \begin{cases} 1, & 1 \leq n \leq 68, \\ 2T(\lfloor n/2 \rfloor + 17) + n, & n > 68. \end{cases}$$

求 $T(n)$ 的渐近紧界。提示: 尝试通过换元消去子问题规模中的常数 17。

习题 7.3 (终止阈值非 1 的情况). 设 $M \geq 1$ 为整数, 递推式为

$$T(n) = \begin{cases} 8T(n/2) + 1, & n^2 > M, \\ M, & n^2 \leq M. \end{cases}$$

求 $T(n)$ 。

习题 7.4 (重复调用与变步长循环). 下面的程序中, 每次调用 Calc 都花费 $\Theta(1)$ 时间。设 N 为正整数, 求 $\text{Fun}(1, N)$ 的时间复杂度紧界。

算法 7.3 $\text{Fun}(l, r)$

```

1:  $n \leftarrow r - l + 1$ 
2: if  $n \leq 1234$  then
3:   return
4: end if
5:  $m \leftarrow \lfloor (l + r)/2 \rfloor$ ;  $m_1 \leftarrow \lfloor (l + m)/2 \rfloor$ ;  $m_2 \leftarrow \lfloor (m + 1 + r)/2 \rfloor$ 
6:  $\text{Fun}(l, m)$ ;  $\text{Fun}(m_1 + 1, m_2)$ 
7: for  $k = 1, \dots, n$  do
8:   for  $i = 1, \dots, n$  do
9:     for  $j = l, l + i, l + 2i, \dots$ , 不超过  $r$  do
10:       $\text{Calc}(j, i)$ 
11:    end for
12:   end for
13: end for
14:  $\text{Fun}(m + 1, r)$ ;  $\text{Fun}(m_1 + 1, m_2)$ 

```

习题 7.5 (三分搜索). 设非空数组 $A[1..n]$ 先严格递减、后严格递增, 且最低点唯一 (允许位于端点)。请设计三分搜索算法, 并分析时间复杂度。提示: 要通过三分搜索找到最低点, 设当前搜索区间为 $[l, r]$, 并取 $m_1 = l + \lfloor (r - l)/3 \rfloor$, $m_2 = l + \lfloor 2(r - l)/3 \rfloor$ 。考虑当 $A[m_1] < A[m_2]$ 、 $A[m_1] > A[m_2]$ 或两者相等时, 应分别保留哪个区间?

习题 7.6 (选择问题分组大小). 把确定性选择算法中每五个元素一组改为每七个元素一组, 其他步骤保持不变, 证明算法仍有 $O(n)$ 的时间上界。如果改为每三个元素一组, 能够得到什么时间上界?

7.5 参考答案

习题 7.1. 第一项正确, 代入法即可证明; 第二项错误, $O(n^2)$ 只给出了附加成本的上界, 没有说明它至少需要多少时间: 例如, 若附加成本恒为 1, 递推式为 $T(n) = 7T(n/3) + 1$, 其解是 $\Theta(n^{\log_3 7})$, 达不到二次量级; 第三项正确, 应用定理 7.8 即可。□

习题 7.2. 令 $m = n - 34$, $S(m) = T(m + 34)$, 则换元后的递推式为

$$S(m) = \begin{cases} 1, & 1 \leq m \leq 34, \\ 2S(\lfloor m/2 \rfloor) + m + 34, & m > 34. \end{cases}$$

当 $m > 34$ 时, $m \leq m + 34 \leq 2m$, 所以附加时间为 $\Theta(m)$, 故 $S(m) = \Theta(m \log m)$, 最后代回 $m = n - 34$ 有 $T(n) = S(n - 34) = \Theta(n \log n)$ 。□

习题 7.3. 如果 $n^2 \leq M$, 根结点就直接返回, $T(n) = M$ 。如果 $n^2 > M$, 第一次满足终止条件的深度是 $d = \lceil \log_2(n/\sqrt{M}) \rceil$ 。深度 d 处共有 8^d 个叶子, 每个叶子的成本为 M 。此前第 i 层有 8^i 个非叶结点, 每个成本为 1。因此

$$T(n) = M8^d + \sum_{i=0}^{d-1} 8^i = M8^d + \frac{8^d - 1}{7} = \Theta\left(\frac{n^3}{\sqrt{M}}\right).$$

□

习题 7.4. 先计算一次调用中的循环成本。固定 i 后, j 从 l 开始, 每次增加 i , 一直到不超过 r 为止, 因此 Calc 被调用 $\lfloor (n-1)/i \rfloor + 1 = \lceil n/i \rceil$ 次。对 $i = 1, \dots, n$ 求和, 利用 $n/i \leq \lceil n/i \rceil < n/i + 1$, 得到 $n \sum_{i=1}^n \frac{1}{i} \leq \sum_{i=1}^n \lceil n/i \rceil < n \sum_{i=1}^n \frac{1}{i} + n$ 。调和级数的渐近界为 $\Theta(\log n)$, 所以这两层循环合计为 $\Theta(n \log n)$ 。最外层的 k 循环再执行 n 次, 整段循环的成本就是 $\Theta(n^2 \log n)$ 。

再看递归部分。程序共调用四次 Fun, 各个区间长度都为 $n/2 + O(1)$, 故 $T(n) = 4T(n/2) + \Theta(n^2 \log n)$, 由定理 7.8, 初始调用 Fun(1, N) 的时间复杂度为 $\Theta(N^2 \log^2 N)$ 。□

习题 7.5. 如果 $A[m_1] < A[m_2]$, 最低点不可能位于 m_2 或其右侧, 保留 $[l, m_2]$ 即可; 如果 $A[m_1] > A[m_2]$, 保留 $[m_1, r]$; 如果 $A[m_1] = A[m_2]$, 最低点必在它们之间; 剩余不超过三个元素时直接比较并返回最小值即可。这一算法每次更新都保留了最低点, 并把区间长度缩小到原来的约 $2/3$, 递推式为 $T(n) = T(2n/3) + O(1)$, 时间复杂度为 $O(\log n)$ 。□

习题 7.6. 改为 7 个元素一组: $T(n) \leq T(n/7 + O(1)) + T(5n/7 + O(1)) + cn$, 不难证明 $O(n)$ 的上界; 改为 3 个元素一组: $T(n) \leq T(n/3 + O(1)) + T(2n/3 + O(1)) + cn$, 得到 $O(n \log n)$ 的上界。□

7.6 参考资料

1. 浙江大学《高级数据结构与算法分析》课程课件第七讲。
2. Thomas H. Cormen 等, 算法导论, 第三版, 第 4 章、第 9.3 节及第 30.1–30.2 节: 最大子数组、Strassen、递推式证明、最坏线性时间选择、多项式乘法与 FFT。
3. Mark Allen Weiss, *Data Structures and Algorithm Analysis in C*, 第二版, 第 10.2 节: 分治运行时间、最近点对、选择与乘法。
4. Jon Kleinberg, Éva Tardos, *Algorithm Design*, 第 5.3–5.5 节: 逆序对、最近点对、整数乘法。
5. Tom Leighton, *Notes on Better Master Theorems for Divide-and-Conquer Recurrences*, 1996。
6. José C. Pereira, Fernando G. Lobo, *An Optimized Divide-and-Conquer Algorithm for the Closest-Pair Problem in the Planar Case*, 2012, *Journal of Computer Science and Technology* 27(4), 891–896。